

CUDA Exercises

M. Ronzani

October 12, 2025

Disclaimer: this PDF has been automatically generated from a Jupiter notebook, expect formatting issues.

1 CUDA

Exploiting NVIDIA GPUs for general-purpose computing.

1.1 Glossary

Thread: one running instance of the kernel, with its own execution context.

Warp: group of threads that are scheduled together on a streaming multiprocessor and get SIMD (or, as Nvidia calls it, SIMT) execution when their program counters align.

Block: group of threads that get assigned to the same streaming multiprocessor, sharing shared memory and synchronization. A block internally consists of one or more warps.

Grid: group of blocks that defines the full launch configuration of a kernel.

Note: Warps are decided by the hardware, blocks and grid are user defined.

Full math for a thread's ID inside a block:

```
thread_id = threadIdx.x +  
            threadIdx.y * blockDim.x +  
            threadIdx.z * blockDim.x * blockDim.y
```

ID of the block the current thread is part of:

```
block_id = blockIdx.x +  
           gridDim.x * blockIdx.y +  
           gridDim.x * gridDim.y * blockIdx.z
```

Absolute (grid-level) thread ID and position:

```
global_thread_id = block_id * (blockDim.x * blockDim.y * blockDim.z) + thread_id
```

```
global_x_pos = blockIdx.x * blockDim.x + threadIdx.x  
global_y_pos = blockIdx.y * blockDim.y + threadIdx.y  
global_z_pos = blockIdx.z * blockDim.z + threadIdx.z
```

Thread idx inside a warp and warp ID in a block:

```
idx_in_warp = thread_id % warpSize
```

```
warp_id = thread_id / warpSize
```

Dimensions are projected into a linearized row-major (x before y before z) order before partitioning into warps.

Note: for a block whose size is not a multiple of warpSize, the last warp will be padded with extra threads to fill up the warpSize-thread positions. For example, with warpSize = 32, if a block has 48 threads, it will be partitioned into two warps, and its second warp will be padded with 16 extra threads.

Available variables to each thread:

- `blockIdx(.x|.y|.z)`: idx of the thread's block
- `blockDim(.x|.y|.z)`: size of one block (threads per dimension)
- `threadIdx(.x|.y|.z)`: idx of the thread in its block
- `gridDim(.x|.y|.z)`: size of the grid of blocks (blocks per dimension)
- `warpSize`: number of threads per warp (implementation detail)

Note: the type `dim3` is a 3D uint vector with default value 1 for unspecified dimensions.

Note: grid size = number of blocks, block size = number of threads per block

Launching a kernel (2D example):

```
// you choose threads_per_row, threads_per_col
dim3 blockDim(threads_per_row, threads_per_col);
// given in_rows, in_cols for the data you need to process, infer the required grid
dim3 gridDim((in_rows + blockDim.x - 1)/blockDim.x, (in_cols + blockDim.y - 1)/blockDim.y);
// e.g.: one shared value per thread in the block
unsigned int sharedMemory = blockDim.x*blockDim.y*sizeof(...);
kernelFunction<&&<numblocks, blocksize,="" sharedmemory="">&&>(...);
cudaDeviceSynchronize();
```

All those “weird” sums before divisions are a fast way to compute the ceiling with integers:

Let $n, m \in \mathbb{N}$, then $\text{ceil}(n/m) = \text{floor}((n + m - 1)/m)$.

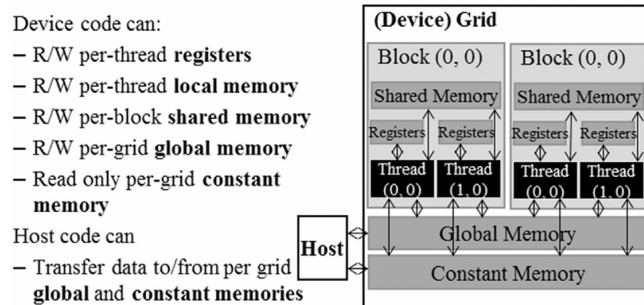
We can also write it as $\lceil \frac{n}{m} \rceil = \lfloor \frac{n+m-1}{m} \rfloor$.

Then, recall that ‘floor’ is automatically applied by the division between unsigned integers in C/C++. In truth, what happens is ‘truncation’, so for negative numbers it equates to applying ‘ceil’. But for index arithmetics this matters not, as it’s almost always positive. Therefore, by using the right side of the above equalities, we don’t need to compute anything beside an integer sum and division, no ceil/floor required.

Available attributes:

- `__global__`: marks a function (kernel) callable from the host with the `<<<...>>>` syntax
- `__shared__`: marks memory that is shared by all threads of a block (vital for inter-thread **data re-use**)
 - `extern __shared__` the shared memory size is decided at kernel launch (last entry in `<<<...>>>`)
- `__device__`: marks a function or variable that lives on the device
- `__constant__`: marks constant memory, i.e., cached, read-only memory on the device
- `__host__`: marks a function as a host function (runs on the CPU), implied by default
- `__host__ __device__`: marks a function callable from both device and host (one version is compiled for each)
- `__managed__`: declares a variable in unified memory, accessible from both host and device, with automatic migration
- `__restrict__`: tells the compiler that a pointer does not alias with others, allowing better optimization (this is C++, not CUDA-specific)

The CUDA memory model:



CUDA memory hierarchy overview.

Credit: “Programming Massively Parallel Processors” by W. Hwu, D. Kirk, I. Hajj

CUDA gives you small vector data types:

- double2
- double3
- double4
- float2
- float3
- float4
- half: 16bit float
- half2
- bfloat16: 8bit exponent (same as float), 7+1bit mantissa (low precision)
- bfloat162

Additional useful stuff:

- `__syncthreads()`: used inside a kernel to create a barrier (typical case: load data in shared memory -> sync -> compute on data)
- `int __syncthreads_count(predicate)`: sync and count how many instances of “predicate” are true among all threads
- `int __syncthreads_and(predicate)`, `int __syncthreads_or(predicate)`: same as above, but return the AND or OR of predicates
- `int __all(predicate)`: same as `__syncthreads_and`, but among threads in the same warp (used to manually speedup branches)
- `int __any(predicate)`: same as `__syncthreads_or`, but among threads in the same warp
- atomic operations: `atomicAdd`, `atomicSub`, `atomicMin`, `atomicMax`, `atomicInc` (increment), `atomicDec` (decrement), `atomicExch` (exchange), `atomicCAS` (compare-and-swap), `atomicAnd` (bitwise), `atomicOr`, `atomicXor`
- `__threadfence_block()`: wait until all global and shared memory writes are visible to all threads in the same block
- `__threadfence()`: wait until all shared memory writes are visible to all threads in block and global memory writes are visible to all threads

Note: the CAS is very powerful to create global atomic locks!

```

// global variable: 0 unlocked, 1 locked
__device__ int lock = 0;
__global__ void kernel(...) {
    ...
    if (threadIdx.x == 0) {
        // acquire lock
        do {} while(atomicCAS(&lock, 0, 1));
        ...
        __threadfence();
        // release lock
        lock = 0;
    }
}

```

Warp shuffles:

- `T __shfl_up_sync(unsigned mask, T var, unsigned int delta)`, retrieves the value of `var` from another thread in the same warp that has `warp_id delta` lower than the caller
 - `mask` controls which threads are involved, usually set to -1 or 0xffffffff (every thread in the warp)
 - `var` is the local register variable from which to take the values to shuffle around
 - `delta` is the offset within the warp where to go fetch the value, if the appropriate thread does not exist (i.e. it's off the end of the warp) then the value is taken from the current thread
 - return value retrieved by the current thread
- `T __shfl_down_sync(unsigned mask, T var, unsigned int delta)`, same as above, but the `delta` is added to the caller's `warp_id` instead of subtracted
- `T __shfl_xor_sync(unsigned mask, T var, unsigned int lane_mask)` a XOR operation is performed between `lane_mask` and the calling thread's `warp_id` (or "lane") to determine the thread from which to copy the value (`lane_mask` controls which bits of the caller's id are "flipped")
- `__shfl_sync(unsigned mask, T var, unsigned int src_lane)`, simply go and fetch `var` from the thread with `warp_id src_lane`

Rules of thumb:

- the CUDA mentality is to launch exactly as many thread blocks as will fill the GPU or match the input/output problem shape
- first fix the `blockDim` (# threads per block) then select `gridDim` (# of blocks) to fit the input (or output - depends on the algorithm's degree of parallelism)
- one can determine if a control construct can result in thread divergence by inspecting its decision condition. If the decision condition is based on `threadIdx` values, the control statement can potentially cause thread divergence
- preload data in shared memory when more than one thread in a block accesses (reuses) the same element (ideally many times)
- imagine the mapping of global thread indices to data indices as a multivalued function, if the collective image of the threads in one block overlaps with the image of those in another, you will have duplicate elements in shared memory (this is fine, but you need more memory)

Some general steps to compute control divergence:

- consider a single block, analyze how it is divided in warps
- identify blocks that have divergence

- if the divergence is caused by a boundary condition, those are the blocks on the boundary
- otherwise, you must find the underlying pattern causing the divergence
- for each of these blocks, identify the warps that have divergence
 - do not evaluate every block from scratch, bundle together those seeing the same divergence pattern
- you can now compute the fraction of diverged warps

Note: this notebook sometimes voluntarily changes coding conventions, like using 'i' for IDs instead of 'id'. This is intended to train the reader to understand someone else's code at a glance.

Note: much alike the above, many values that could be 'unsigned int' are written just as 'int' for readability. These kind of micro-optimizations are left to the reader.

Note: nvcc treats all code as C++, not C. Still, this notebook uses as little as possible syntax from C++.

1.2 Exercise 1

Test your CUDA environment (compiler support and available GPU card):

```
[ ]: !nvcc --version
```

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Thu_Jun__6_02:18:23_PDT_2024
Cuda compilation tools, release 12.5, V12.5.82
Build cuda_12.5.r12.5/compiler.34385749_0
```

```
[ ]: !nvidia-smi -L
```

```
GPU 0: Tesla T4 (UUID: GPU-1079b5db-aa98-6348-11c3-c91825cc7fa7)
```

To know about the capabilities of you GPU, you can query the device properties and/or read the [compute capability specifications](#).

Then, you should be able to answer the following questions:

- What are the maximum dimensions of a thread block?
- What is the maximum number of threads in a block?
- What is the maximum size of the GPU global memory?
- What is the warp size?
- How many Streaming Multiprocessors are available?
- Is an execution configuration of <<<100, 1000>>> feasible on this device?
- Suppose you have a CUDA program using multiple blocks, should each block contain 4x4, 8x8, or 30x30 threads?

```
[ ]: %%writefile deviceProperties.cpp
//      ^^^ The extension should be 'cu', here we use 'cpp' to the Colab's UI
↪syntax highlight working...
#include <stdio.h>

int main() {
    int deviceCount;
    cudaGetDeviceCount(&deviceCount);

    for (int dev = 0; dev < deviceCount; dev++){

        cudaDeviceProp deviceProp;
        cudaGetDeviceProperties(&deviceProp, dev);
```

```

printf("Device properties of GPU card ID %d (%s)\n", dev, deviceProp.name);
printf("  Compute capability: %d.%d\n", deviceProp.major, deviceProp.minor);
printf("  Warp size: %d\n", deviceProp.warpSize);
printf("  -----\n");
printf("  Maximum GPU global memory size: %zu bytes\n", deviceProp.totalGlobalMem);
printf("  Maximum GPU constants memory size: %zu bytes\n", deviceProp.totalConstMem);
printf("  Maximum shared memory available per block: %zu bytes\n", deviceProp.
→sharedMemPerBlock);
    printf("  Maximum shared memory available per SM: %zu bytes\n", deviceProp.
→sharedMemPerMultiprocessor);
    printf("  -----\n");
    printf("  Maximum size of each dimension of a grid: %d x %d x %d\n", deviceProp.
→maxGridSize[0], deviceProp.maxGridSize[1], deviceProp.maxGridSize[2]);
    printf("  Maximum size of each dimension of a block: %d x %d x %d\n", deviceProp.
→maxThreadsDim[0], deviceProp.maxThreadsDim[1], deviceProp.maxThreadsDim[2]);
    printf("  Maximum number of threads per block: %d\n", deviceProp.maxThreadsPerBlock);
    printf("  Number of 32-bit registers available per block: %d\n", deviceProp.regsPerBlock);
    printf("  -----\n");
    printf("  Number of Streaming Multiprocessors: %d\n", deviceProp.multiProcessorCount);
    printf("  Maximum number of resident blocks per SM: %d\n", deviceProp.
→maxBlocksPerMultiProcessor);
    printf("  Maximum resident threads per SM: %d\n", deviceProp.maxThreadsPerMultiProcessor);
    printf("  Number of 32-bit registers available per SM: %d\n", deviceProp.
→regsPerMultiProcessor);

}
}

```

Writing deviceProperties.cpp

```

[ ]: !mv deviceProperties.cpp deviceProperties.cu
    !nvcc deviceProperties.cu -run

```

```

Device properties of GPU card ID 0 (Tesla T4)
Compute capability: 7.5
Warp size: 32
-----
Maximum GPU global memory size: 15828320256 bytes
Maximum GPU constants memory size: 65536 bytes
Maximum shared memory available per block: 49152 bytes
Maximum shared memory available per SM: 65536 bytes
-----
Maximum size of each dimension of a grid: 2147483647 x 65535 x 65535
Maximum size of each dimension of a block: 1024 x 1024 x 64
Maximum number of threads per block: 1024
Number of 32-bit registers available per block: 65536
-----
Number of Streaming Multiprocessors: 40
Maximum number of resident blocks per SM: 16
Maximum resident threads per SM: 1024
Number of 32-bit registers available per SM: 65536

```

1.3 Exercise 2

Hardware questions, if a streaming multiprocessor (SM) has resource for:

- 32 threads per warp

- 2048 (2^{11}) threads
- 65536 (2^{16}) registers
- at most 255 registers per thread

At maximum occupancy, how many registers does each execution context have?

Up to how many threads can execute the same instruction simultaneously (on different data)?

Note: use more registers than are left in the SM and it will cause spilling of registers to local memory, significantly slowing threads down.

These above are the specs for an A100, that has 108 SMs.

Suppose we have a kernel requesting 1000 blocks, each with 128 threads. How does it get executed?

On current hardware, we would probably get 8-12 blocks running at the same time on each SM, and each block has 4 warps, resulting in 32-48 warps running on each SM.

In any case, a SM can support at most 32 simultaneous blocks.

Assume now a device that allows up to 32 blocks per SM and 2048 threads per SM. Indicate which of the following assignments per SM are possible. In the cases in which it is possible, indicate the occupancy level. 1) 8 blocks with 128 threads each 2) 32 blocks with 32 threads each 3) 64 blocks with 32 threads each 4) 32 blocks with 64 threads each

The same device has 65536 (2^{16}) registers per SM. Assume that the kernel being run uses 128 registers per thread and no additional memory. Indicate which of the following assignments per SM are possible. In the cases in which it is possible, indicate the occupancy level. 1) 8 blocks with 128 threads each 2) 16 blocks with 32 threads each 3) 32 blocks with 32 threads each 4) 8 blocks with 64 threads each

Note: underoccupancy can also arise from registers being oversubscribed. Registers are a fairly limited resource in modern GPUs!

1.4 Exercise 3

Question 1:

For a vector addition (element-wise sum), assume that the vector length is 8000, each thread calculates one output element, and the thread block size is 1024 threads. The programmer configures the kernel launch to have a minimal number of thread blocks to cover all output elements.

How many threads will be in the grid?

Question 2:

We want to use each thread in a kernel to calculate two (adjacent) output elements of a vector addition. That is, we use half as many threads as vector elements.

What would be the expression for mapping the thread/block indices to data index for the two elements to be added?

Question 3:

We want to use each thread in a kernel to calculate two output elements of a vector addition. Each thread block processes $2 \times \text{blockDim.x}$ consecutive elements that form two sections. All threads in each block will first process a section, each processing one element. They will then all move to the next section, again each processing one element. You can assume blockDim.x is a multiple of two.

What would be the expression for mapping the thread/block indices to data index for the two elements to be added?

If 'n' is the length of the vector and the thread block size is 1024 threads, what does the execution configuration ('<<<...>>>') look like?

Question 4:

A kernel is launched with '<<<(16, 16), (24, 12)>>>' on a device with $\text{warpSize} = 32$.

Write down the threadIdx.x and threadIdx.y at which the 2nd and 3rd warp of each block start and end.

Question 5:

A kernel is launched with '<<<(16, 8, 16), (6, 4, 8)>>>' on a device with warpSize = 32.

Write down the threadIdx.x, threadIdx.y, and threadIdx.z at which the 1st and 2nd warp of each block start and end.

Recall: threads are always assigned to warps in row-major order, with x fastest, then y, then z.

1.5 Exercise 4

We are to process an 800x600 picture (800 pixels in width/x, 600 pixels in height/y) with the following dumbPictureKernel().

That is m = 600 and n = 800, with one thread per pixel.

Assume it is executed with blocks of 16x16 threads and warps of 32 threads: 1) How many warps will be generated during the execution of the kernel? 2) How many warps will have control divergence? 3) With m = 800 and n = 599, how many warps have control divergence? 4) With m = 799 and n = 600, how many warps have control divergence?

```
[ ]: %%writefile only_for_highlight.cpp

__global__
void dumbPictureKernel(float* d_Pin, float* d_Pout, int n, int m) {
    // calculate the row idx of the d_Pin and d_Pout element to process
    int row = blockIdx.y*blockDim.y + threadIdx.y;
    // calculate the column idx of the d_Pin and d_Pout element to process
    int col = blockIdx.x*blockDim.x + threadIdx.x;

    // each thread computes one element of d_Pout if in range
    if ((row < m) && (col < n)) {
        d_Pout[row*n + col] = 2*d_Pin[row*n + col];
    }
}
```

This example is more common than you may think:

A typical reason for using a control construct with thread divergence is handling boundary conditions when mapping threads to data. This is because the total number of threads needs to be a multiple of the block size whereas the size of the data can be an arbitrary number.

Fortunately, as the input size grows (1D - linearly, 2D - quadratically, 3D - cubically), the boundary becomes smaller (1D - no growth, 2D - linearly, 3D - quadratically). Think how the perimeter of a square is always less than its area, even moreso when the sidelength grows. This means that, as we scale up, divergence affects a smaller fraction of warps, resulting in a lower overhead percentage.

1.6 Exercise 5

Given the following vector addition program, estimate what impact will have the serialization due to control divergence (aka warp divergence). Assume warps of 32 threads.

Quantify:

- the percentage of warps encountering divergence
- the SIMD efficiency for the addition operation in the kernel (# threads doing it in parallel / # threads not doing it in parallel) in each diverged warp

```
[ ]: %%writefile vecadd2.cpp
#include<stdio.h>

__global__
void vecAddKernel(float* A, float* B, float* C, int n) {
```



```

int i = blockDim.x*blockIdx.x + threadIdx.x;
if (i < n)
    C[i] = A[i] + B[i];
}

int main() {
    int n = 1000;

    // timing
    float milli;
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    float *a;
    float *b;
    float *c;

    cudaMallocManaged(&a, n*sizeof(float));
    cudaMallocManaged(&b, n*sizeof(float));
    cudaMallocManaged(&c, n*sizeof(float));

    // arbitrary initialization
    for (float i = 0; i < n; i++) {
        a[i] = i;
        b[i] = i + 5;
    }

    cudaEventRecord(start);
    vecAddKernel<<<4, 256>>>(a, b, c, n);
    cudaEventSynchronize(stop);
    cudaEventRecord(stop);
    cudaEventElapsedTime(&milli, start, stop);

    // unfortunately, this won't pick up on the slight overhead of divergence...
    printf("Execution time (ms): %f \n", milli);

    cudaEventDestroy(stop);
    cudaFree(a);
    cudaFree(b);
    cudaFree(c);

    return 0;
}

```

Writing vecadd2.cpp

```

[ ]: !mv vecadd2.cpp vecadd2.cu
    !nvcc -arch=sm_75 vecadd2.cu -run

```

Execution time (ms): 0.000000

How divergence occurs:

- small branch (like above): all threads execute both branches with masking
- big branches (10+ instructions each): the compiler inserts “voting instructions” to speedup the case where all threads go in the same branch, if that does not happen, the cost is again the sum of both branches

Branches are handled per-warp, so the maximum throughput penalty is 1/32x on a A100 if only one thread goes down

a different path in every warp.

Workarounds:

- determine at compile time that all threads in the warp must go the same way (thank you very much)
- if you can know a priori which part of the input takes which branch, create two or more kernels, each executing one branch, and launch them on their respective inputs

Example:

Processing a long list of elements where, depending on run-time values, a few require very expensive processing.

GPU implementation:

First process list to build two sub-lists of “simple” and “expensive” elements, then process the two sub-lists separately. You need 3 kernels, one per element type and one to filter the list.

Note: none of this is new, this is what we did 35 years ago on CRAY machines.

1.7 Exercise 6

Given a vector, double the value of its elements in even position, while put to zero the elements in odd position if they are less than 1.0, otherwise leave them unchanged.

This kernel implements the optimization proposed in Exercise 5. Therefore

Questions: 1) what would have been the impact of control divergence if only we have only a single kernel doing both tasks with a `if` checking if the current index was odd/even? Assume warps of 32 threads. 2) could we do the same “split the kernel in one per branch” trick to avoid divergence due to the `if(v[i] < 1.0)` check?

```
[ ]: %%writefile vecadd2.cpp
#include<stdio.h>

__global__
void evenElements(float* v, int n) {
    int i = blockDim.x*2*blockIdx.x + threadIdx.x*2;
    if (i < n)
        v[i] = v[i]*2;
}

__global__
void oddElements(float* v, int n) {
    int i = blockDim.x*2*blockIdx.x + threadIdx.x*2 + 1;
    if (i < n)
        if (v[i] < 1.0)
            v[i] = 0.0;
}

int main() {
    // HP: let n always be even
    int n = 1000;
    int numThreads = (n + 1)/2;
    int blockSize = 16;
    int numBlocks = (numThreads + blockSize - 1)/blockSize;

    float *v;
    cudaMallocManaged(&v, n*sizeof(float));
    for (float i = 0; i < n; i++)
        v[i] = i/10;

    evenElements<<<numBlocks, blockSize>>>(v, n);
```

```

oddElements<<<numBlocks, blockSize>>>(v, n);
cudaDeviceSynchronize();

cudaFree(v);
return 0;
}

```

```

[ ]: !mv vecadd2.cpp vecadd2.cu
      !nvcc -arch=sm_75 vecadd2.cu -run

```

A big catch:

Since per-thread work is extremely tiny, in this case the divergence penalty is very small. On the other hand, doing two global sweeps, complete with kernel launch and all, adds enough overhead to dominate over the benefit of not diverging.

While this trick of creating multiple kernel is useful, reserve it for when the amount of work along either branch is quite substantial.

1.8 Exercise 7

Given the following two kernels, estimate what impact will have the serialization due to control divergence (assume warps of 32 threads).

In both kernels, we assign to an output element the maximum that can be found between its matching input position and a certain number of positions ahead.

```

[ ]: %%writefile only_for_highlight.cpp

#define N = 1024
#define D = 16

__global__ void maxInNextElements(float* v, float* o, int n) {
    int g_id = blockIdx.x*blockDim.x + threadIdx.x;
    int limit = min(D, gridDim.x*blockDim.x - g_id);

    float res = 0.0;
    for (int i = 0; i < limit; i += 1) {
        res = max(res, v[g_id + i]);
    }
    o[g_id] = res;
}

// launched with:
int main() {
    float v[N], o[N];
    //...
    maxInNextElements<<<4, N/4>>>(v, o, N);
    //...
}

```

```

[ ]: %%writefile only_for_highlight.cpp

#define N = 1024
#define D = 16

__global__ void maxInNextElements(float* v, float* o, int n) {
    int g_id_x = blockIdx.x*blockDim.x + threadIdx.x;
    int g_id_y = blockIdx.y*blockDim.y + threadIdx.y;
}

```

```

int g_id_z = blockIdx.z*blockDim.z + threadIdx.z;
int limit_x = min(D, gridDim.x*blockDim.x - g_id_x);
int limit_y = min(D, gridDim.y*blockDim.y - g_id_y);
int limit_z = min(D, gridDim.z*blockDim.z - g_id_z);

float res = 0.0;
for (int i = 0; i < limit_x; i += 1) {
    int x = g_id_x + i;
    for (int j = 0; j < limit_y; j += 1) {
        int y = g_id_y + j;
        for (int k = 0; k < limit_z; k += 1) {
            int z = g_id_z + k;
            res = max(res, v[x + y*N + z*N*N]);
        }
    }
}
o[g_id_x + g_id_y*N + g_id_z*N*N] = res;
}

// launched with:
int main() {
    float v[N*N*N], o[N*N*N];
    //...
    maxInNextElements<<<<(4, 4, 4), (N/4, N/4, N/4)>>>>(v, o, N);
    //...
}

```

1.9 Exercise 8

The following kernel takes a vector and updates each element as the average of itself, its two following neighbors, and its two preceding neighbors. If an element is on the boundary of the vector, outside elements are considered to be 0.

Crucially, the kernel is sped-up by loading data in shared memory for reuse among threads in the same block. The canonical term to refer to reuse in this case is “halo”, because if you imagine to run the kernel sequentially over the vector, left to right, as you move you would be continuously reusing 4-out-of-5 elements that you just used for the immediately prior computation, that is, your input at each vector position continuously overlaps with the “lingering shadow” of that of the previous one.

Be wary that the shared memory allocation needs to be slightly larger than the number of elements handled by the block. It needs to store 4 extra values, 2 below and 2 above the current block.

```

[ ]: %%writefile only_for_highlight.cpp
#include<stdio.h>

__global__
void neighborsAverage(const float* __restrict__ in, float* __restrict__ out, int n) {
    extern __shared__ float s[];
    int t_id = threadIdx.x;
    int g_t_id = blockIdx.x * blockDim.x + t_id;

    // load into shared memory with halo of 2 on each side
    int l_idx = t_id + 2; // make space for the first two elements
    if (g_t_id < n)
        s[l_idx] = in[g_t_id];
    else
        s[l_idx] = 0.0f;

    // load left halo

```

```

if (t_id < 2) {
    int left_idx = g_t_id - 2 + t_id;
    s[t_id] = (left_idx >= 0) ? in[left_idx] : 0.0f;
}
// load right halo
if (t_id >= blockDim.x - 2) {
    int right_idx = g_t_id + (t_id - (blockDim.x - 2)) + 1;
    s[l_idx + 2] = (right_idx < n) ? in[right_idx] : 0.0f;
}

__syncthreads();

// 5 elements average
if (g_t_id < n) {
    out[g_t_id] = (s[l_idx-2] + s[l_idx-1] + s[l_idx] + s[l_idx+1] + s[l_idx+2]) / 5.0f;
}
}

int main() {
    const int n = 4096;
    float h_in[n], h_out[n];
    for (int i = 0; i < n; i++)
        h_in[i] = i + 1;

    float *d_in, *d_out;
    cudaMalloc(&d_in, n*sizeof(float));
    cudaMalloc(&d_out, n*sizeof(float));
    cudaMemcpy(d_in, h_in, n*sizeof(float), cudaMemcpyHostToDevice);

    int blockSize = 256;
    int gridSize = (n + blockSize - 1)/blockSize;
    size_t shmem = (blockSize + 4) * sizeof(float); // +4 for halos
    neighborsAverage<<<gridSize, blockSize, shmem>>>(d_in, d_out, n);
    cudaMemcpy(h_out, d_out, n*sizeof(float), cudaMemcpyDeviceToHost);

    for (int i = 0; i < n; i++)
        printf("%.2f ", h_out[i]);
    printf("\n");

    cudaFree(d_in);
    cudaFree(d_out);
    return 0;
}

```

If, as shown, we launch the kernel with blocks of 256 threads, and the vector has length 4096, what is the impact of control divergence?

Note: here divergence arises from the extra loads in shared memory!

1.10 Exercise 9

Consider the following code snippet, representing a matrix multiplication kernel:

```

[ ]: %%writefile only_for_highlight.cpp

// HP: square matrices of side length 'n'
__global__
void MatrixMulKernel(float* A, float* B, float* C, int n) {

```

```

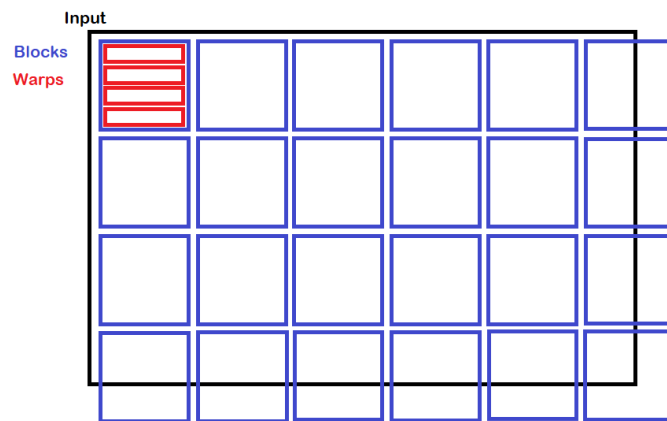
int row = blockIdx.y*blockDim.y + threadIdx.y;
int col = blockIdx.x*blockDim.x + threadIdx.x;
if ((row < n) && (col < n)) {
    float partial_out = 0;
    for (int k = 0; k < n; ++k) {
        partial_out += A[row*n + k] * B[k*n + col];
    }
    C[row*n + col] = partial_out;
}
}

```

If we launch the kernel with a block size of 16x16 on a 1000x1000 matrix, how many warps will have control divergence? Assume warpSize = 64 (fictional, but you never know...).

With an SM that can support up to 1,536 threads and up to 8 thread blocks, which of the following block configuration would result in the most number of threads in each SM: 64, 128, 512 or 1024 threads per block?

A little visual help:



2D visualization of the input, blocks, and warps.

Note: ops, "input" should have been "output", well, too late for that.

1.11 Exercise 10

Question 1:

Assume that each atomic operation in a DRAM system has a total latency of 100ns. What is the maximal throughput we can get for atomic operations on the same global memory variable?

Assuming now that the kernel does 5 floating-point operations per atomic operation, what is the maximal floating-point throughput of the kernel execution as limited by the throughput of the atomic operations?

Question 2:

For a GPU that supports atomic operations in L2 cache, assume that each atomic operation takes 4ns to complete in L2 cache and 100ns to complete in DRAM. Assume that 90% of the atomic operations hit in L2 cache. What is the approximate throughput for atomic operations on the same global memory variable?

Question 3:

Continuing from Question 1.

Assume that we privatized some global memory variables into shared memory variables, and the shared memory access latency is 1ns. For simplicity, assume that the additional global memory atomic operations for accumulating privatized variable into the global variable adds 10% to the total execution time. Assume that a kernel performs 5 floating-point operations per atomic operation. What is the maximal floating-point throughput of the kernel execution as limited by the throughput of the atomic operations?

1.12 Histogram

Given an array whose elements are all from a set known a priori, count the occurrences of each element of the set in the array.

In this example, let the set be the ASCII letters from 'a' to 'z', and the array length be 'n'.

Let's also add the concept of "bins", these are subset of elements that count towards the same total. For us, letters will be binned in groups of four, that is, 'a-d' will count towards the same total, then 'e-h', and so on.

Key issues to solve:

- data access locality
- output interference

1.12.1 Wrong (poor memory access efficiency - and data races)

Each thread is given a contiguous section of the input.

The input array is evenly partitioned and given to threads, each thread then consumes its partition one element at a time.

Issue:

At a given point in time, all threads will be asking to access values from memory that sit at very distant locations one from the other. We cannot bundle this data in a single DRAM transaction.

```
[ ]: %%writefile only_for_highlight.cpp

__global__
void histKernel(unsigned char *arr, unsigned int *hist, long n) {
    int part_size = (n + gridDim.x*blockDim.x - 1)/gridDim.x*blockDim.x;
    int tid = (blockIdx.x*blockDim.x + threadIdx.x)*part_size;

    for (int i = tid; i < tid + part_size && i < n; i++) {
        int letter_idx = arr[i] - 97; // 97 is the ASCII code of 'a'
        if (letter_idx >= 0 && letter_idx < 26)
            hist[letter_idx/4] += 1; // divide by 4 to get bins
    }
}

int main() {
    // HP: less threads than elements
    long n = 10000;
    int blockSize = 16;
    int numBlocks = 32;

    unsigned char *arr;
    unsigned int *hist;
    cudaMallocManaged(&arr, n*sizeof(float));
    cudaMallocManaged(&hist, 7*sizeof(float)); // 26 letters in bins of 4 -> 7 bins
    for (int i = 0; i < n; i++)
        arr[i] = (unsigned char)(i%26 + 97);
    for (int i = 0; i < 7; i++)
        hist[i] = 0;

    histKernel<<<numBlocks, blockSize>>>(arr, hist, n);
    cudaDeviceSynchronize();

    cudaFree(arr);
    cudaFree(hist);
}
```

```

return 0;
}

```

1.12.2 Wrong (data races)

Threads process the array in an interleaved fashion.

With the whole grid of threads being smaller than the array, the first thread in the grip processes the first element, the second the second, and so on. Then, the whole grid is “moved” to the right by its own size, therefore the first thread will process element $|grid|+0$, the second $|grid|+1$, and so on. They will then move to $2*|grid|+0$, $2*|grid|+1$, ...

Why?

This way, at any point in time, all threads process a **contiguous section of elements**. Therefore, **memory accesses are coalesced**, because threads step forward together, and ask for contiguous data all at once. Ultimately, this gives better spatial locality -> multiple piece of data transferred in one DRAM transaction.

There is still one issue: updates to the histogram can happen however they please. This means that two threads could read the same value for ‘ $k = hist[x]$ ’ and thus both write ‘ $k + 1$ ’, thereby losing one of the two increments.

This is a classic problem of writing shared memory parallel code: the **data race**.

```

[ ]: %%writefile only_for_highlight.cpp

__global__
void histKernel(unsigned char *arr, unsigned int *hist, long n) {
    int tid = blockIdx.x*blockDim.x + threadIdx.x;
    int stride = blockDim.x*gridDim.x;

    for (int i = tid; i < size; i += stride) {
        int letter_idx = arr[i] - 97;
        if (letter_idx >= 0 && letter_idx < 26)
            hist[letter_idx/4] += 1;
    }
}

// same main as above...

```

1.12.3 Atomics

To prevent concurrent updates (read-modify-write operations) to cause data races, we just need to give a thread exclusive access to the modified value from before it reads it until it finishes the update. Essentially, an atomic context.

CUDA gives use ready-to-use primitives for this, at least for most basic operations, therefore, instead of a normal ‘+1’ on the histogram, we need to perform its atomic equivalent ‘atomicAdd(...)’.

Issue:

Atomic operations are semantically correct, but take ~1000 cycles to complete on DRAM, , this severely hurts performance.

So, could we get away without atomic operations while preserving correctness? Yes, and as it is often the case, we just need to postpone some part of the program! In this case, we will postpone the global accumulation of histogram bin counts.

```

[ ]: %%writefile only_for_highlight.cpp

__global__
void histKernel(unsigned char *arr, unsigned int *hist, long n) {
    int tid = blockIdx.x*blockDim.x + threadIdx.x;
    int stride = blockDim.x*gridDim.x;

```



```

for (int i = tid; i < size; i += stride) {
    int letter_idx = arr[i] - 97;
    if (letter_idx >= 0 && letter_idx < 26)
        atomicAdd(&(hist[letter_idx/4]), 1);
}
}

// same main as above...

```

1.12.4 Privatization

Create a copy of the histogram counters per block, using shared memory.

Then, rely on atomics on shared memory, since they take ~10 cycles to complete on L2 cache, that is 100x better than DRAM.

As the very last step, accumulate the counters from all copies.

```

[ ]: %%writefile only_for_highlight.cpp

__global__
void histKernel(unsigned char *arr, unsigned int *hist, long n, unsigned int num_bins) {
    int tid = blockIdx.x*blockDim.x + threadIdx.x;
    int stride = blockDim.x*gridDim.x;
    extern __shared__ unsigned int hist_s[];

    for(unsigned int bin_idx = threadIdx.x; bin_idx < num_bins; bin_idx += blockDim.x)
        hist_s[bin_idx] = 0u;

    __syncthreads();

    // compute histogram in shared memory
    for (unsigned int i = tid; i < n; i += stride) {
        int letter_idx = arr[i] - 97;
        if (letter_idx >= 0 && letter_idx < 26)
            atomicAdd(&(hist_s[letter_idx/4]), 1);
    }

    __syncthreads();

    // accumulate between blocks
    for(unsigned int bin_idx = threadIdx.x; bin_idx < num_bins; bin_idx += blockDim.x)
        atomicAdd(&(hist[bin_idx]), hist_s[bin_idx]);
}

// same main as above, just pass '7' as 'num_bins'.

```

All major problems are now solved! The code could still see some minor optimization, like performing a reduction between the private histogram copies, instead of relying again on atomics, but that is left to the reader.

1.13 Extras

1.13.1 Occupancy Calculation

Several API functions exist to assist programmers in choosing thread block size and cluster size based on register and shared memory requirements.

Here we showcase the “occupancy calculator” API, `cudaOccupancyMaxActiveBlocksPerMultiprocessor`, can provide an occupancy prediction based on the block size and shared memory usage of a kernel. This function reports occupancy

in terms of the number of concurrent thread blocks per multiprocessor.

```
[ ]: %%writefile occupancy_calculator.cpp
__global__
void MyKernel(int *d, int *a, int *b) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    d[idx] = a[idx] * b[idx];
}

int main() {
    int numBlocks; // occupancy in terms of active blocks
    int blockSize = 32;
    // these variables are used to convert occupancy to warps
    int device;
    cudaDeviceProp prop;
    int activeWarps;
    int maxWarps;

    cudaGetDevice(&device);
    cudaGetDeviceProperties(&prop, device);
    cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocks, MyKernel, blockSize, 0);

    activeWarps = numBlocks * blockSize / prop.warpSize;
    maxWarps = prop.maxThreadsPerMultiProcessor / prop.warpSize;
    std::cout << "Occupancy: " << (double)activeWarps / maxWarps * 100 << "%" << std::endl;

    return 0;
}
```

1.13.2 Yet Another Simple Exercise on Divergence

A tiled matrix multiplication kernel has many boundary condition checks, such as the ones to load M tiles in the following code snippet:

```
[ ]: %%writefile only_for_highlight.cpp

#define TILE_WIDTH 16

if (Row < Width && t*TILE_WIDTH + tx < Width) {
    ds_M[ty][tx] = M[Row*Width + p*TILE_WIDTH + tx];
} else {
    ds_M[ty][tx] = 0.0;
}
```

Assuming 100x100 square matrices, 16x16 square tiles, and 16x16 thread blocks, estimate what will be the overall impact of control divergence in this case.

1.13.3 An Improbable Divergence Scenario

Given the following kernel, estimate what impact will have the serialization due to control divergence.

```
[ ]: %%writefile only_for_highlight.cpp

#define N 1024

__global__ void loopKernel(float* v, float* o, int n) {
    int g_id = blockIdx.x*blockDim.x + threadIdx.x;
    int delta = g_id % 2;
```

```

    o[g_id] = 0;
    for (int i = -delta; i <= delta; i += 1) {
        o[g_id] += 2*v[g_id + i];
    }
}

// launched with:
int main() {
    float v[N], o[N];
    //...
    loopKernel<<<4, N/4>>>(v, o, N);
    //...
}

```

This code is very unlikely, but for the sake of an example, think of it as a weird take on a 1D convolution.

How to improve it up is obvious: write two kernels, one for even and one for odd position elements!

```

[ ]: %%writefile only_for_highlight.cpp

#define N 1024

__global__ void loopKernelEven(float* v, float* o, int n) {
    int g_id = (blockIdx.x*blockDim.x + threadIdx.x)*2;

    o[g_id] = 0;
    for (int i = -1; i <= 1; i += 1) {
        o[g_id] += 2*v[g_id + i];
    }
}

__global__ void loopKernelOdd(float* v, float* o, int n) {
    int g_id = (blockIdx.x*blockDim.x + threadIdx.x)*2 + 1;

    o[g_id] += 2*v[g_id];
}

// launched with:
int main() {
    float v[N], o[N];
    //...
    loopKernelEven<<<4, N/8>>>(v, o, N);
    loopKernelOdd<<<4, N/8>>>(v, o, N);
    //...
}

```

1.13.4 Warp Shuffles for Reduction

Let us improve the reduction!

- classic version: each block moves a part of the input in shared memory, reduces it one logarithmic tree level at a time (with syncs in between), and then does the final reduction across blocks;
- shuffle version: each block moves a part of the input in shared memory, each warp reduces internally its set of values, then we go at the block level, and finally across blocks;

```

[ ]: %%writefile reduce.cpp
#include <stdio.h>

```

```

#include <stdlib.h>

#define WARP_SIZE 32

// classic shared-memory reduction kernel
__global__
void blockReduceClassic(const float* __restrict__ g_in, float* __restrict__ g_out, size_t n) {
    extern __shared__ float sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int idx = blockIdx.x * blockDim.x * 2 + tid;

    float val = 0.0f;
    if (idx < n)
        val = g_in[idx];
    if (idx + blockDim.x < n)
        val += g_in[idx + blockDim.x];

    sdata[tid] = val;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) // active threads: first 's' of each block
            sdata[tid] += sdata[tid + s];
        __syncthreads();
    }

    if (tid == 0)
        g_out[blockIdx.x] = sdata[0];
}

// warp shuffle helper
__inline__ __device__
float warpReduceSum(float val) {
    unsigned mask = 0xffffffffu; // all 1s, all active
    for (int offset = warpSize / 2; offset > 0; offset >>= 1)
        val += __shfl_down_sync(mask, val, offset);
    return val;
}

// shuffle-based reduction kernel
__global__
void blockReduceShuffle(const float* __restrict__ g_in, float* __restrict__ g_out, size_t n) {
    extern __shared__ float sdata[];
    unsigned int tid = threadIdx.x;
    unsigned int lane = tid % warpSize; // position (lane) inside warp
    unsigned int warp_id = tid / warpSize;

    size_t idx = (size_t)blockIdx.x * blockDim.x * 2 + tid;

    float sum = 0.0f;
    if (idx < n)
        sum = g_in[idx];
    if (idx + blockDim.x < n)
        sum += g_in[idx + blockDim.x];

    sum = warpReduceSum(sum);
}

```

```

if (lane == 0)
    sdata[warp_id] = sum;
__syncthreads();

if (warp_id == 0) {
    float val = (lane < (blockDim.x / warpSize)) ? sdata[lane] : 0.0f;
    val = warpReduceSum(val);
    if (lane == 0)
        g_out[blockIdx.x] = val;
}
}

// shuffle version (host entry point)
float reduceShuffle(const float* d_in, size_t n) {
    int threads = 1024;
    int blocks = (int)((n + threads * 2 - 1) / (threads * 2)); // 2 elements per thread => each
    thread does 1 sum in the first level
    float* d_out;
    cudaMalloc(&d_out, blocks * sizeof(float));

    size_t shmem = (threads / WARP_SIZE) * sizeof(float);
    blockReduceShuffle<<<blocks, threads, shmem>>>(d_in, d_out, n);

    float* h_out = (float*)malloc(blocks * sizeof(float));
    cudaMemcpy(h_out, d_out, blocks * sizeof(float), cudaMemcpyDeviceToHost);

    float sum = 0.0f;
    for (int i = 0; i < blocks; i++)
        sum += h_out[i];

    free(h_out);
    cudaFree(d_out);
    return sum;
}

// classic version (host entry point)
float reduceClassic(const float* d_in, size_t n) {
    int threads = 1024;
    int blocks = (int)((n + threads * 2 - 1) / (threads * 2));
    float* d_out;
    cudaMalloc(&d_out, blocks * sizeof(float));

    size_t shmem = threads * sizeof(float);
    blockReduceClassic<<<blocks, threads, shmem>>>(d_in, d_out, n);

    float* h_out = (float*)malloc(blocks * sizeof(float));
    cudaMemcpy(h_out, d_out, blocks * sizeof(float), cudaMemcpyDeviceToHost);

    float sum = 0.0f;
    for (int i = 0; i < blocks; i++)
        sum += h_out[i];

    free(h_out);
    cudaFree(d_out);
    return sum;
}

```

```

int main() {
    size_t N = 1 << 30;
    float* h_in = (float*)malloc(N * sizeof(float));
    for (size_t i = 0; i < N; i++)
        h_in[i] = 1.0f;

    float* d_in;
    cudaMalloc(&d_in, N * sizeof(float));
    cudaMemcpy(d_in, h_in, N * sizeof(float), cudaMemcpyHostToDevice);

    // timing
    float milli;
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start);
    float sumClassic = reduceClassic(d_in, N);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaEventElapsedTime(&milli, start, stop);
    printf("Execution classic time (ms): %f \n", milli);

    cudaEventRecord(start);
    float sumShuffle = reduceShuffle(d_in, N);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaEventElapsedTime(&milli, start, stop);
    printf("Execution shuffle time (ms): %f \n", milli);

    printf("Computed values:\nExpected: %.0f\n", (float)N);
    printf("Classic:  %.0f\n", sumClassic);
    printf("Shuffle:  %.0f\n", sumShuffle);

    cudaEventDestroy(start);
    cudaEventDestroy(stop);
    cudaFree(d_in);
    free(h_in);
    return 0;
}

```

Writing reduce.cpp

```

[ ]: !mv reduce.cpp reduce.cu
     !nvcc -arch=sm_75 reduce.cu -run

```

```

Execution classic time (ms): 85.974594
Execution shuffle time (ms): 35.604031
Computed values:
Expected: 1073741824
Classic:  1073741824
Shuffle:  1073741824

```

1.13.5 Jacobi Iteration Method for the Laplace Equation

The Laplace equation is a partial differential equation (PDE) in the form:

$$\nabla^2 u(x, y) = 0 \Leftrightarrow \frac{\delta^2 u}{\delta x^2} + \frac{\delta^2 u}{\delta y^2} = 0$$

And arises in many physical contexts, like steady-state heat conduction and fluid flow.

We want to solve it numerically, and for that, we discretize x and y in a uniform grid with spacing h , and we approximate the laplacian at each interior point via finite differences w.r.t. its neighbors:

$$\nabla^2 u(x_i, y_j) \approx \frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}}{h^2}$$

Imposing $\nabla^2 u(x, y) = 0$ thus gives us a simple dependency of each point on its four neighbors:

$$u_{i,j} = \frac{1}{4}(u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1})$$

In other words, we have a solution when the value at each interior point equals the average of its four neighbors.

The **Jacobi method** is a simple iterative technique to approximate solutions of linear systems, and we will use it to find the steady-state above.

Applied to the Laplace equation:

- start with an initial guess $u^{(0)}$
- at each iteration, compute a new solution $u^{(k+1)}$ by replacing every interior grid point with the average of its neighbors from the previous iteration
- keep the boundary values fixed (these are the Dirichlet boundary conditions)
- continue until convergence to a steady-state

```
[ ]: %%writefile jacobi.cpp
#include <stdio>
#include <stdlib>
#include <math>

__global__
void lap(int I, int J, const float* __restrict__ u1, float* __restrict__ u2) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    int j = threadIdx.y + blockIdx.y * blockDim.y;

    if (i >= I || j >= J)
        return;
    int id = i + j * I;

    if (i == 0 || i == I - 1 || j == 0 || j == J - 1) {
        // boundary: Dirichlet condition, keep fixed
        u2[id] = u1[id];
    } else {
        // interior: average of neighbors
        u2[id] = 0.25f * (u1[id - 1] + u1[id + 1] +
            u1[id - I] + u1[id + I]);
    }
}

int main() {
    const int I = 128;          // grid width
    const int J = 128;          // grid height
    const int N = I * J;        // total points
    const int max_iter = 500;    // fixed iteration count

    float *h_u = (float*)malloc(N * sizeof(float));

    // initialization: boundary = 0, interior = random values
    for (int j = 0; j < J; j++) {
        for (int i = 0; i < I; i++) {
            int id = i + j * I;
```

```

        if (i == 0 || i == I-1 || j == 0 || j == J-1)
            h_u[id] = 0.0f;
        else
            h_u[id] = static_cast<float>(rand()) / RAND_MAX;
    }
}

float *d_u1, *d_u2;
cudaMalloc(&d_u1, N * sizeof(float));
cudaMalloc(&d_u2, N * sizeof(float));

cudaMemcpy(d_u1, h_u, N * sizeof(float), cudaMemcpyHostToDevice);

dim3 blockDim(16, 16);
dim3 gridDim((I + blockDim.x - 1)/blockDim.x, (J + blockDim.y - 1)/blockDim.y);

// Jacobi iterations
for (int iter = 0; iter < max_iter; iter++) {
    lap<<<gridDim, blockDim>>>(I, J, d_u1, d_u2);
    cudaDeviceSynchronize();

    // NOTE: we can't update locations in place, since other threads will be using them at
    ↳the same time!
    // => swap pointers (ping-pong)
    float* tmp = d_u1;
    d_u1 = d_u2;
    d_u2 = tmp;
}

cudaMemcpy(h_u, d_u1, N * sizeof(float), cudaMemcpyDeviceToHost);

printf("Result snapshot:\n");
for (int j = 0; j < 5; j++) {
    for (int i = 0; i < 5; i++) {
        printf("%6.3f ", h_u[i + j * I]);
    }
    printf("\n");
}

free(h_u);
cudaFree(d_u1);
cudaFree(d_u2);

return 0;
}

```

```

[ ]: !mv jacobi.cpp jacobi.cu
      !nvcc -arch=sm_75 jacobi.cu -run

```

Extra:

Check for convergence, every K Jacobi iterations run a second kernel that checks for convergence between ‘d_u1’ and ‘d_u2’. The check simply consist of finding, across all locations, the one with the maximum difference between ‘d_u1’ and ‘d_u2’, if that difference is less than a user-define tolerance, then terminate. In other words, this is just a reduce for the maximum value.


```

[ ]: %%writefile highlight_only.cpp

#define K 10

__global__
void max_diff(int N, const float* u1, const float* u2, float* d_out) {
    __shared__ float sdata[256];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + tid;

    float diff = 0.0f;
    if (idx < N)
        diff = fabsf(u1[idx] - u2[idx]);
    sdata[tid] = diff;
    __syncthreads();

    // this is the classic shared memory reduce
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            if (sdata[tid + stride] > sdata[tid])
                sdata[tid] = sdata[tid + stride];
        }
        __syncthreads();
    }

    if (tid == 0) d_out[blockIdx.x] = sdata[0];
}

int main() {
    // ... skipping code from before ...

    float tolerance = 1e-4f;
    float *d_blockmax, *h_blockmax;
    int blocks = (N + 255) / 256;
    h_blockmax = (float*)malloc(blocks * sizeof(float));
    cudaMalloc(&d_blockmax, blocks * sizeof(float));

    for (int iter = 0; iter < max_iter; iter++) {
        lap<<<gridDim, blockDim>>>(I, J, d_u1, d_u2);
        cudaDeviceSynchronize();

        // every K iterations, convergence check
        if (iter % K == 0) {
            max_diff<<<blocks, 256>>>(N, d_u1, d_u2, d_blockmax);
            cudaMemcpy(h_blockmax, d_blockmax, blocks*sizeof(float), cudaMemcpyDeviceToHost);

            // finish the reduce on the host
            float maxdiff = 0.0f;
            for (int b = 0; b < blocks; b++)
                if (h_blockmax[b] > maxdiff)
                    maxdiff = h_blockmax[b];

            if (maxdiff < tolerance) {
                printf("Converged at iteration %d with diff=%e\n", iter, maxdiff);
                break;
            }
        }
    }
}

```

```
// swap pointers
float* tmp = d_u1;
d_u1 = d_u2;
d_u2 = tmp;
}

// ... skipping code from before ...
}
```